

Java Backend Interview Prep - JPA, JDBC, and Database

This document answers the JPA/JDBC and Database topics listed in the source interview sheet.
[file:1]

JPA and Spring Data JPA

1) JPA vs Hibernate vs MyBatis vs Spring Data JPA

- **JPA:** specification/API for ORM.
- **Hibernate:** popular JPA implementation with extra features.
- **MyBatis:** SQL mapper, not full ORM.
- **Spring Data JPA:** abstraction on top of JPA repositories to reduce boilerplate.[file:1]

2) transient keyword

`transient` prevents a field from Java serialization. In JPA discussion, interviewers may also expect `@Transient`, which excludes a field from persistence mapping.

3) Employee entity example

```
import jakarta.persistence.*;

@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false)
    private int age;

    @Column(nullable = false)
    private double salary;

    public Employee() {}
}
```

4) @GeneratedValue strategies

- AUTO: provider decides.
- IDENTITY: database auto-increment/identity column.
- SEQUENCE: database sequence.
- TABLE: separate table stores IDs.[file:1]

5) AUTO strategy behavior

AUTO delegates strategy selection to the JPA provider/dialect. Choice depends on database capabilities and provider defaults.[file:1]

6) SEQUENCE vs IDENTITY

SEQUENCE is often preferred when supported because it enables better batching and allocation optimization. IDENTITY can limit batch insert efficiency because IDs are usually known only after insert.[file:1]

7) SequenceGenerator and TableGenerator

They define generator metadata like backing sequence/table name and allocation size.

8) Custom ID generator

Use provider-specific custom generators or UUID-based strategies when domain requirements need custom formats.

9) JpaRepository vs CrudRepository

JpaRepository extends CrudRepository and adds JPA-specific features like flushing, batch-related methods, and paging/sorting convenience through related interfaces.[file:1]

10) @Query with JPQL and modifying queries

Use @Query for custom JPQL/native SQL. For update/delete queries, add @Modifying and execute inside a transaction.[file:1]

```
@Modifying
@Transactional
@Query("update Employee e set e.salary = :salary where e.id = :id")
int updateSalary(@Param("id") Long id, @Param("salary") double salary);
```

11) Pagination and sorting

Use Pageable and Sort.

```
Page<Employee> page = repo.findByDepartmentId(10L, PageRequest.of(0, 20, Sort.by("sa.
```

12) EntityManager role

EntityManager manages persistence context, entity lifecycle, dirty checking, JPQL/native query execution, and transactions at the JPA level.[file:1]

13) Relationship mappings

- @OneToOne: one entity linked to one.
- @OneToMany: one parent to many children.
- @ManyToOne: many children to one parent.
- @ManyToMany: many on both sides.

14) LAZY vs EAGER

- LAZY: load relation on access.
- EAGER: load immediately.

Defaulting blindly to EAGER is dangerous because it can inflate queries and object graphs.[file:1]

15) N+1 query problem

N+1 happens when one query loads parents and then additional queries are fired for each child/association access. It can appear with lazy loading or badly designed eager fetching depending on usage pattern.[file:1]

16) Auditing

Use @CreatedDate, @LastModifiedDate, and enable JPA auditing.

17) Optimistic vs pessimistic locking

- **Optimistic:** detect conflicts using version column, good for read-heavy systems.
- **Pessimistic:** lock rows in DB, useful when conflicts are frequent.

```
@Version  
private Long version;
```

18) Multi-data-source and multi-tenancy

For multiple data sources, define separate `DataSource`, `EntityManagerFactory`, and transaction manager beans. Multi-tenancy options include database-per-tenant, schema-per-tenant, or shared-schema with discriminator.[file:1]

19) Native queries and projections

Spring Data JPA supports interface-based and DTO projections with JPQL or native queries for optimized reads.[file:1]

20) Bidirectional relationship JSON recursion

Prevent infinite recursion using DTOs, `@JsonManagedReference`/`@JsonBackReference`, or `@JsonIgnore` depending on API shape.[file:1]

21) Cascade operations

Common cascade types: `PERSIST`, `MERGE`, `REMOVE`, `REFRESH`, `DETACH`, `ALL`.

22) Entity graphs

Entity graphs control which associations should be fetched for a query, helping avoid N+1 while not over-fetching everything.[file:1]

23) Batch inserts/updates

Use JDBC batching/Hibernate batch settings, sequence-based IDs, and periodic flush/clear for large datasets.[file:1]

24) NamedQuery vs NamedNativeQuery

- `@NamedQuery`: JPQL.
- `@NamedNativeQuery`: native SQL.

25) @Query precedence

When both are available for the same repository method, an explicit `@Query` typically takes precedence over derived or named query resolution.[file:1]

26) Index definition in JPA

```
@Entity
@Table(name = "employees", indexes = {
    @Index(name = "idx_emp_dept_salary", columnList = "department_id,salary")
})
class Employee { }
```

27) @Index vs unique=true

@Index improves lookup and join/query performance. unique=true enforces uniqueness constraint; it may create a unique index underneath depending on database behavior.[file:1]

Database and SQL

28) DDL vs DML, DELETE vs TRUNCATE, WHERE vs HAVING

- **DDL:** schema commands like CREATE, ALTER, DROP.
- **DML:** data commands like INSERT, UPDATE, DELETE.
- **DELETE:** row-by-row, can use WHERE, transaction-log heavier.
- **TRUNCATE:** removes all rows quickly, usually resets storage/identity behavior depending on DB.
- **WHERE:** filters rows before grouping.
- **HAVING:** filters after grouping.[file:1]

29) INNER JOIN vs LEFT JOIN

- **INNER JOIN:** only matching rows.
- **LEFT JOIN:** all rows from left table plus matching right rows, otherwise nulls.

30) ACID

- Atomicity
- Consistency
- Isolation
- Durability

31) SQL pagination

Use LIMIT/OFFSET or database-specific pagination/windowing.

```
SELECT *  
FROM employee  
ORDER BY salary DESC  
LIMIT 10 OFFSET 20;
```

32) View vs materialized view

- **View:** virtual query definition.
- **Materialized view:** stores computed results physically and needs refresh.

33) SQL execution order

Logical order:[file:1]

FROM -> JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT/OFFSET

34) Second highest salary

```
SELECT name, salary
FROM employee e
WHERE 1 = (
    SELECT COUNT(DISTINCT salary)
    FROM employee
    WHERE salary > e.salary
);
```

35) Employees earning more than department average

```
SELECT e.*
FROM employee e
JOIN (
    SELECT department_id, AVG(salary) avg_salary
    FROM employee
    GROUP BY department_id
) d ON e.department_id = d.department_id
WHERE e.salary > d.avg_salary;
```

36) Top 3 highest paid employees per department

```
SELECT *
FROM (
    SELECT e.*, DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) rnk
    FROM employee e
) x
WHERE rnk <= 3;
```

37) Partitioning types

Range, list, and hash partitioning are common. Sub-partitioning combines strategies for very large tables.[file:1]

38) Horizontal vs vertical partitioning

- **Horizontal:** split rows.
- **Vertical:** split columns.

39) Keys

- Primary key: chosen unique identifier.
- Candidate key: any possible unique identifier.
- Surrogate key: artificial/generated key.
- Composite key: multiple columns together identify a row.[file:1]

A table can exist without a primary key, but updates, deduplication, joins, and integrity management become harder.